

QAOA (2)

QAOAAnsatz() 파헤치기

2026.07.07

2025113574 권나현

System Software Lab.

목차

1. 주제 선정 이유
2. QAOAAnsatz()란?
3. QAOAAnsatz() 구현 (simulator)
4. 결과 해석

1. 주제 선정 이유

- 지난 시간에는 QAOA의 정의와 전체적인 구조, 구현 위주로 탐구하였음
- 이번 시간에는 더 나아가서 QAOA를 수행하기 위한 양자컴퓨팅적 작동 원리를 자세히 파악하고 결과를 해석함으로써 응용할 수 있는 역량을 목표함
- QAOAAnsatz()

-review-

QAOA(Quantum Approximate Optimization Algorithm)의 정의

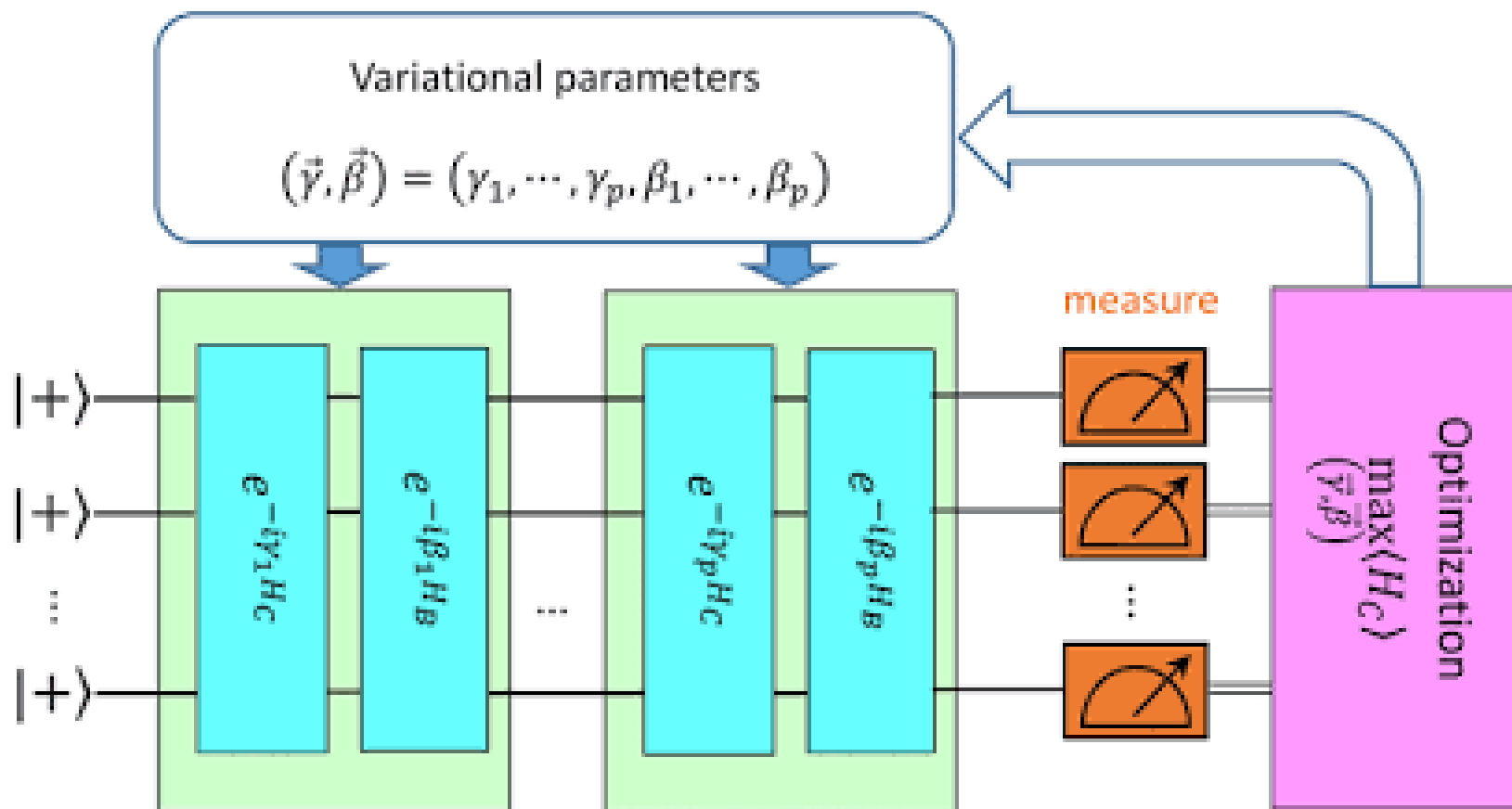
: 양자 근사 최적화 알고리즘, QUBO 문제에 양자컴퓨터를 적용한 고전-양자 하이브리드 알고리즘

QAOA의 원리

비용이 가장 낮은 최적의 답을 에너지가 가장 낮은 바닥 상태와 일치하도록 양자 시스템을 설계하여 양자 컴퓨터가 비용 해밀토니안의 바닥 상태를 찾아내면 최적화 문제의 정답이 된다.

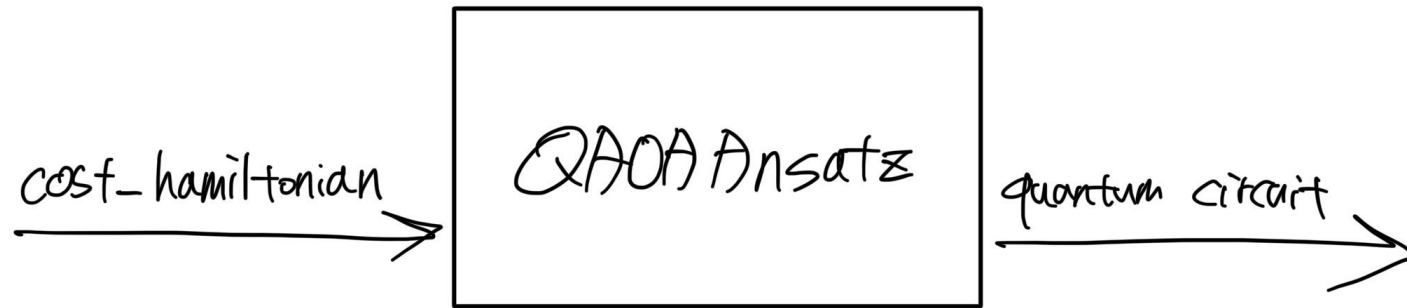
1. 주제 선정 이유

-review-



2. QAOAAnsatz()란?

매개변수가 포함된 양자 회로를 자동으로 만들어주는 함수



review – 비용 해밀토니안?
: 구하고 싶은 비용함수(QUBO 등)를
양자계의 에너지 식(해밀토니안)으로 그대로 모방해서 설계한 것

2. QAOAAnsatz()란?

-QAOAAnsatz()의 사용 예-

```
n_small = 5

graph = rx.PyGraph()
graph.add_nodes_from(np.arange(0, n_small, 1)) # 노드 그리기

# 간선 정보 리스트
abc = [
    (0, 1, 1.0),
    (0, 2, 1.0),
    (0, 4, 1.0),
    (1, 2, 1.0),
    (2, 3, 1.0),
    (3, 4, 1.0),
]
```

cost Hamiltonian 빌드

```
def build_max_cut_paulis(graph):
    pauli_list = []
    for edge in list(graph.edge_list()):
        weight = graph.get_edge_data(edge[0], edge[1])
        pauli_list.append(("ZZ", [edge[0], edge[1]], weight))
    print(pauli_list)
    return pauli_list

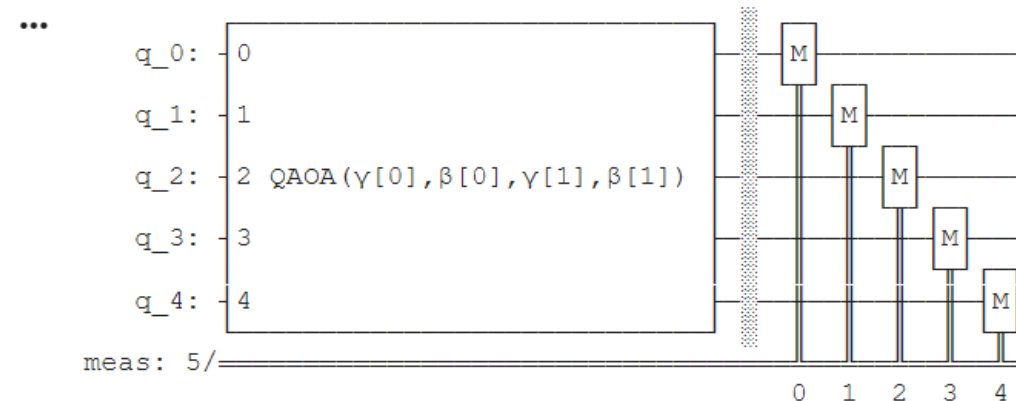
max_cut_paulis = build_max_cut_paulis(graph)
cost_hamiltonian = SparsePauliOp.from_sparse_list(max_cut_paulis, n_small) # Qiskit은 리틀 엔디안 방식을 사용
print("Cost Function Hamiltonian: ", cost_hamiltonian)

... [(('ZZ', [0, 1], 1.0), ('ZZ', [0, 2], 1.0), ('ZZ', [0, 4], 1.0), ('ZZ', [1, 2], 1.0), ('ZZ', [2, 3], 1.0), ('ZZ', [3, 4], 1.0))
Cost Function Hamiltonian: SparsePauliOp([['IIIZZ', 'IIZIZ', 'ZIIIZ', 'IIZZI', 'IZZII', 'ZZIII'],
      coeffs=[1.+0.j, 1.+0.j, 1.+0.j, 1.+0.j, 1.+0.j, 1.+0.j])
```

QAOA ansatz 회로 생성하기

```
▶ circuit = QAOAAnsatz(cost_operator=cost_hamiltonian, reps=2)
circuit.measure_all()

circuit.draw()
```



이 함수만 사용하면 비용 해밀토니안에 대응되는 양자 회로를 자동으로 만들어준다.
이 함수를 사용할 수 있다고 해서 양자컴퓨팅적 원리를 과연 이해했다고 할 수 있을까?

3. QAOAAnsatz() 구현 (simulator)

quantum gate를 사용하여 양자 회로를 직접 설계하면 QAOAAnsatz() 없이도 동일한 기능을 구현할 수 있다.

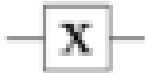
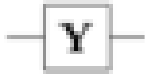
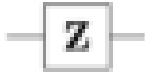
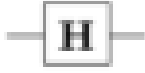
참고: 기본적인 Quantum gates

I gate: Identity

X gate: Bit-flip

Z gate: Phase-flip

Y gate: Bit&Phase flip

Operator	Gate(s)		Matrix
Pauli-X (X)			$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$
Pauli-Y (Y)			$\begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$
Pauli-Z (Z)			$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$
Hadamard (H)			$\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$

3. QAOAAnsatz() 구현 (simulator)

양자 게이트에 대한 추가적인 개념

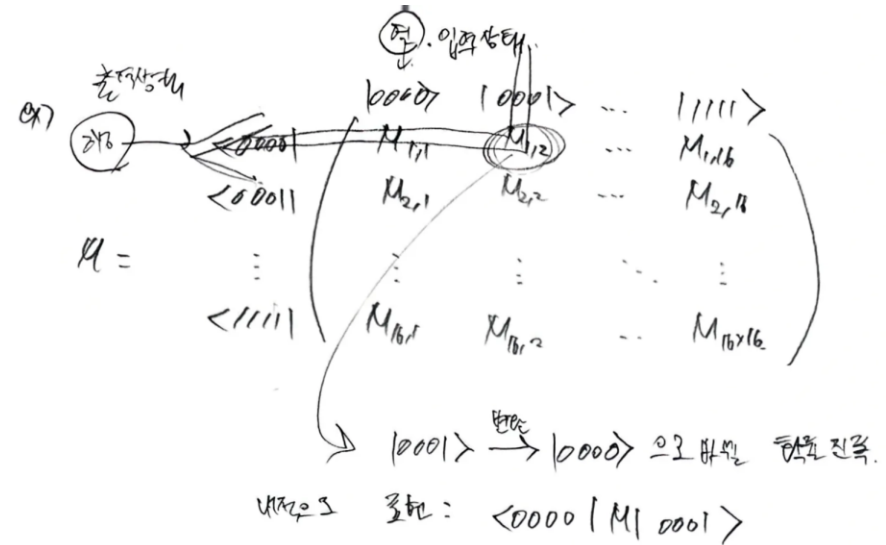
예) 4 큐비트
 $|1000\rangle \sim |1111\rangle$
 $2^{4-1} = 2^4 = 16$ 가지 벡터 → 16×16 Matrix

4 큐비트 양자 상태:
 상태 벡터 $|P\rangle = \begin{pmatrix} C_{0000} \\ C_{0001} \\ \vdots \\ C_{1111} \end{pmatrix}$ (16x1)
 "Column Vector"
 행 벡터 (Row) $\langle P| = \begin{pmatrix} \langle 0000| & \langle 0001| & \dots & \langle 1111| \end{pmatrix}$

→ 양자 게이트 동작 (= Operator), 양자 게이트 (Operator): $|P\rangle$ 를 새로운 $|P\rangle$ 로 변환.

$$\begin{pmatrix} \text{행 벡터} \\ 1 \times 16 \end{pmatrix} \begin{pmatrix} \text{열 벡터} \\ 16 \times 1 \end{pmatrix} = \begin{pmatrix} \text{Matrix} \\ 16 \times 16 \end{pmatrix}$$

↓
 열 (Column): "입력 상태" $|0000\rangle \sim |1111\rangle$
 행 (Row): "출력 상태" $\langle 0000| \sim \langle 1111|$
 양자 게이트의 매개변수 = M 의 테이블
 (입력 A → 출력 B 변환 행렬 (가장치))

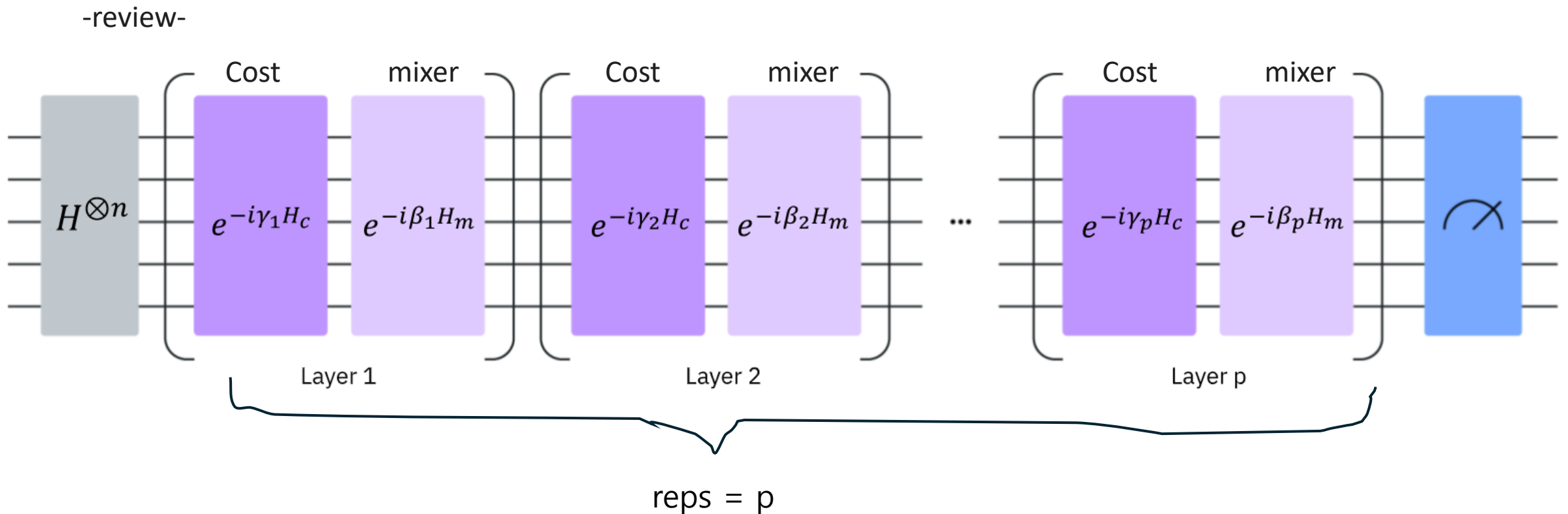


↓
 양자 게이트 M 의 행 벡터 (Row) → 양자 게이트 M 의 열 벡터 (Column)

∴ 4 큐비트
 상태 벡터의 길이 (= 차원, 개수) = 2^4
 행렬 크기: $2^4 \times 2^4$ (양자 게이트)

3. QAOAAnsatz() 구현 (simulator)

QAOA 루프에서 Cost operator와 Mixer operator에 대한 추가적인 개념



3. QAOAAnsatz() 구현 (simulator)

Review - QAOA 루프의 동작 과정

복소수 지수 형태: 회전 변환 행렬을 의미
(어떤 축을 기준으로 상태를 얼마만큼 회전)

1. 초기 중첩 상태 $H^{\otimes n} |0\rangle$: 모든 큐비트에 H 게이트 적용
2. 교대 층(alternating layers): 2개의 레이어를 번갈아가며 반복적으로 적용

1 layer

cost operator:

$$e^{-i\gamma_p H_c}$$

시간(각도) 매개변수
비용 해밀토니안

mixer operator:

$$e^{-i\beta_p H_m}$$

간섭 유발 (상태 섞음)

회전 각도 파라미터

$$\gamma_p, \beta_p$$

는 고전적인 피드백 루프에서 최적화됨

(양자컴퓨터에서는 큐비트를 마이크로펄스로 회전시킨다)

3. QAOAAnsatz() 구현 (simulator)

QAOA 루프에서 Cost operator와 Mixer operator에 대한 추가적인 개념

① 2-body $[(0,1), (0,2), (0,4), (1,2), (2,3), (3,4)]$

→ $(11122), (11212), (21112), (12211), (12211), (22111)$

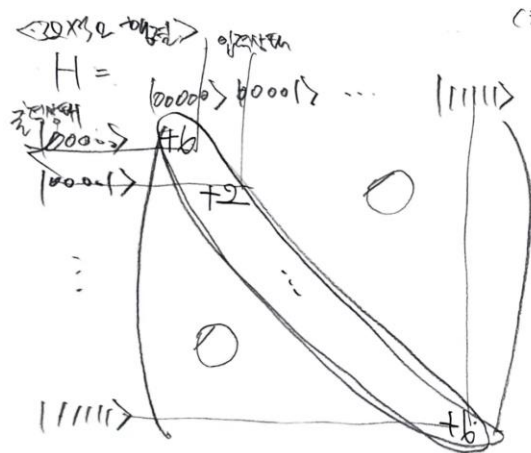
각 항: Pauli-Z 매칭 문제 $\otimes$ $\rightarrow$ 32x32 행렬.

$(0,1)$ 에 ZZ 매칭. $\rightarrow$ $\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \otimes \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \otimes I \otimes I \otimes I$

(2-body 항의 행렬)

$$A_{mn} \otimes B_{pq} = C_{mpnq}$$

$$\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \otimes \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} = \begin{pmatrix} 1 \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} & 0 \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \\ 0 \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} & -1 \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \end{pmatrix}$$



(ZZ 항의 행렬) $\begin{pmatrix} |0\rangle & |1\rangle & |2\rangle & |3\rangle & |4\rangle \end{pmatrix}$

$$= \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix} \quad 4 \times 4$$

$|0\rangle, |1\rangle \rightarrow +1$
 $|0\rangle, |1\rangle \rightarrow -1$

$$H_c = \frac{1}{2} \sum_{(i,j)} (I - Z_i Z_j)$$

$$= \frac{1}{2} (I - Z_0 Z_1 + I - Z_0 Z_2 + I - Z_0 Z_4 + I - Z_1 Z_2 + I - Z_2 Z_3 + I - Z_3 Z_4)$$

$\rightarrow$ 32개

	$(0,1)$	$(0,2)$	$(0,4)$	$(1,2)$	$(2,3)$	$(3,4)$	H
$ 00000\rangle$	+	+	+	+	+	+	+6
$ 00001\rangle$	+	+	-	+	+	-	+2
$\vdots$							
$ 11111\rangle$	+	+	+	+	+	+	+6

H_{ii} = 상태 i 가 가진 에너지

3. QAOAAnsatz() 구현 (simulator)

QAOA 루프에서 Cost operator와 Mixer operator에 대한 추가적인 개념

H_C 가 $\mathbb{U}_C$ 의 지수로 등장 $\rightarrow e^{-i\gamma H_C}$
 $\mathbb{U}_C = e^{-i\gamma H_C} = \begin{pmatrix} e^{-i\gamma f_1} & & & \\ & e^{-i\gamma f_2} & & \\ & & \ddots & \\ & & & e^{-i\gamma f_6} \end{pmatrix}$
 (초기값들 = 학습진폭)

$\langle \text{Mixer} \rangle H_M$: 전항 (비상 상태, 정상 상태 등등)
 (상대 편향) (절대 편향)
 $\rightarrow$ 각 큐비트에 Pauli-X gate 적용하는 연산. (Bit-Flip)
 $\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$
 아래로
 바이트는 이진으로 존재

$H_M = \sum X_j$ ex) 5 큐비트: $H_M = \sum_{j=0}^4 X_j = X_0 + X_1 + X_2 + X_3 + X_4$
 $X_0 I I I I$ $I X I I$ $I I X I$ $I I I X$ $I I I I X$

$$R_{zz}(\theta) = \begin{pmatrix} e^{-i\theta/2} & 0 & 0 & 0 \\ 0 & e^{i\theta/2} & 0 & 0 \\ 0 & 0 & e^{i\theta/2} & 0 \\ 0 & 0 & 0 & e^{-i\theta/2} \end{pmatrix}$$

$$RX(\theta) = \exp\left(-i\frac{\theta}{2}X\right) = \begin{pmatrix} \cos\left(\frac{\theta}{2}\right) & -i\sin\left(\frac{\theta}{2}\right) \\ -i\sin\left(\frac{\theta}{2}\right) & \cos\left(\frac{\theta}{2}\right) \end{pmatrix}$$

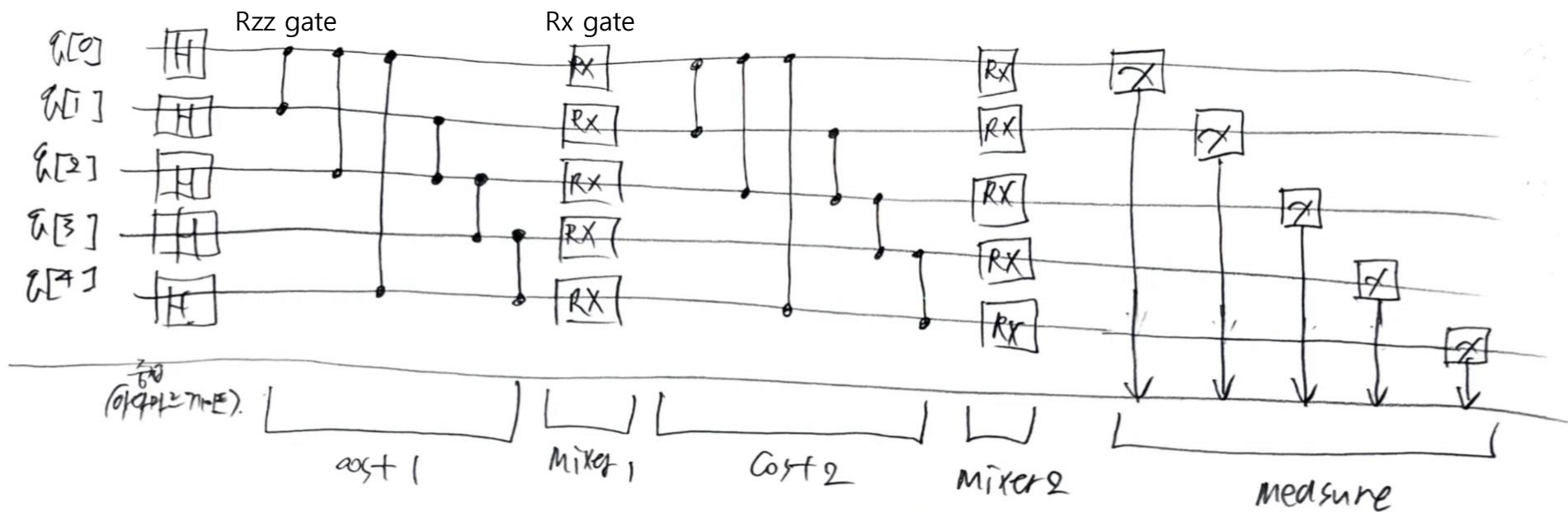
3. QAOAAnsatz() 구현 (simulator)

양자 회로 설계

간선 정보 리스트

```
abc = [  
    (0, 1, 1.0),  
    (0, 2, 1.0),  
    (0, 4, 1.0),  
    (1, 2, 1.0),  
    (2, 3, 1.0),  
    (3, 4, 1.0),  
]
```

```
pauli_list: [('ZZ', [0, 1], 1.0), ('ZZ', [0, 2], 1.0), ('ZZ', [0, 4], 1.0), ('ZZ', [1, 2], 1.0), ('ZZ', [2, 3], 1.0), ('ZZ', [3, 4], 1.0)]  
max_cut_paulis: [('ZZ', [0, 1], 1.0), ('ZZ', [0, 2], 1.0), ('ZZ', [0, 4], 1.0), ('ZZ', [1, 2], 1.0), ('ZZ', [2, 3], 1.0), ('ZZ', [3, 4], 1.0)]  
Cost Function Hamiltonian: SparsePauliOp(['IIIZZ', 'IIZIZ', 'ZIIIZ', 'IIZZI', 'IZZII', 'ZZIII'],  
      coeffs=[1.+0.j, 1.+0.j, 1.+0.j, 1.+0.j, 1.+0.j, 1.+0.j])
```



3. QAOAAnsatz() 구현 (simulator)

```
#circuit = QAOAAnsatz(cost_operator=cost_hamiltonian, reps=2)
from qiskit import QuantumCircuit
from qiskit.circuit import Parameter
from qiskit.circuit import ParameterVector
```

```
circuit = QuantumCircuit(5)
betas = ParameterVector('β', length=2)
gammas = ParameterVector('γ', length=2)
```

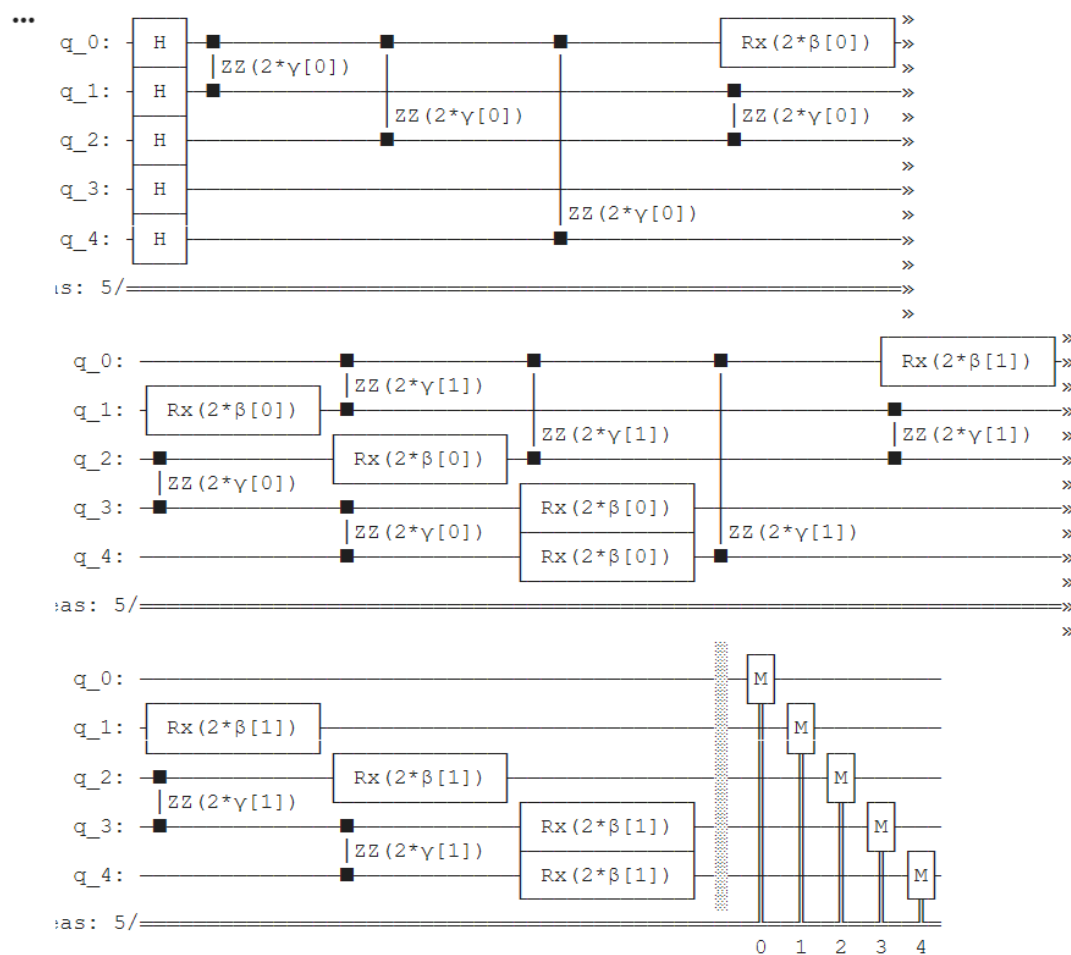
```
# H gate
for i in range(5):
    circuit.h(i)

for i in range(2):
    # cost
    for j in graph.edge_list():
        circuit.rzz(2 * gammas[i], j[0], j[1])
```

```
# mixer
for k in range(5):
    circuit.rx(2 * betas[i], k, label=None)
```

```
circuit.measure_all()
```

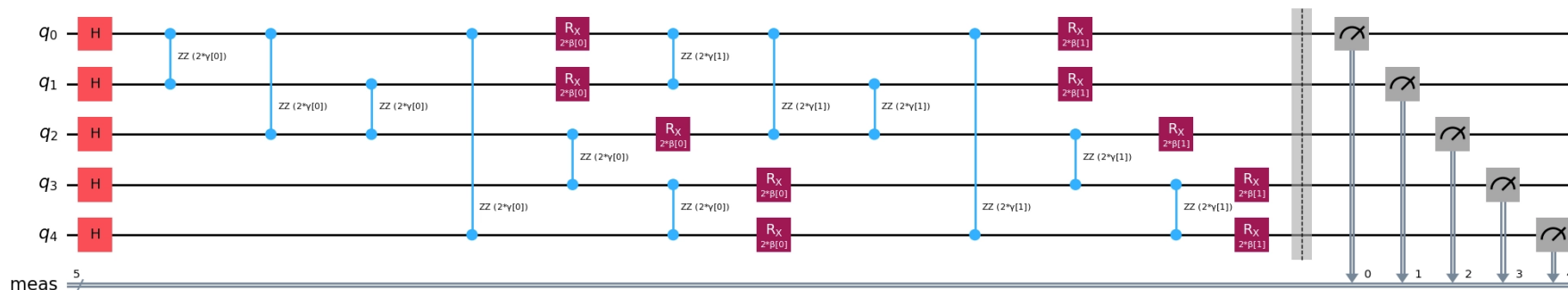
```
circuit.draw()
```



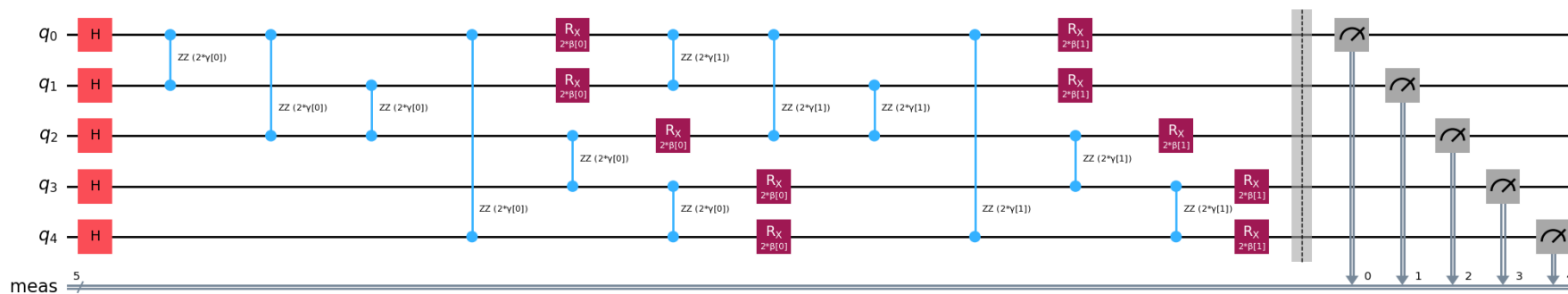
3. QAOAAnsatz() 구현 (simulator)

-회로를 맞게 설계했는지 검증하기: 양자 시뮬레이터-

(QAOAAnsatz 함수를 사용한 코드)



(양자 회로를 직접 설계한 코드)



4. 결과 해석

-결과 해석하는 방법-

ex)

result bitstring: $[0, 0, 1, 0, 1]$

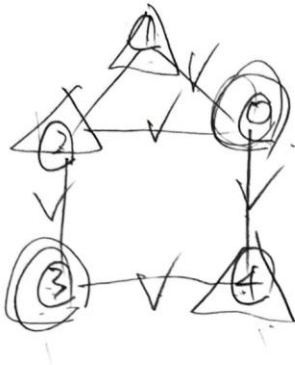
↳ $|00101\rangle$ 의 H값의 가능성

= (대부분의 경우 0이 1과 1이 0이 되는 바깥 상태).

↳ 마다

↳ 그래프 (Bitstrings-reversed)에서 10100 기둥이 가장 높음

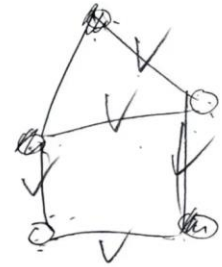
↳ 그래프 (시작점 정해져 있지 X).



두 그룹으로 나뉨

정답은 5.

ex) 1 1 0 1 0



정답은 5

4. 결과 해석

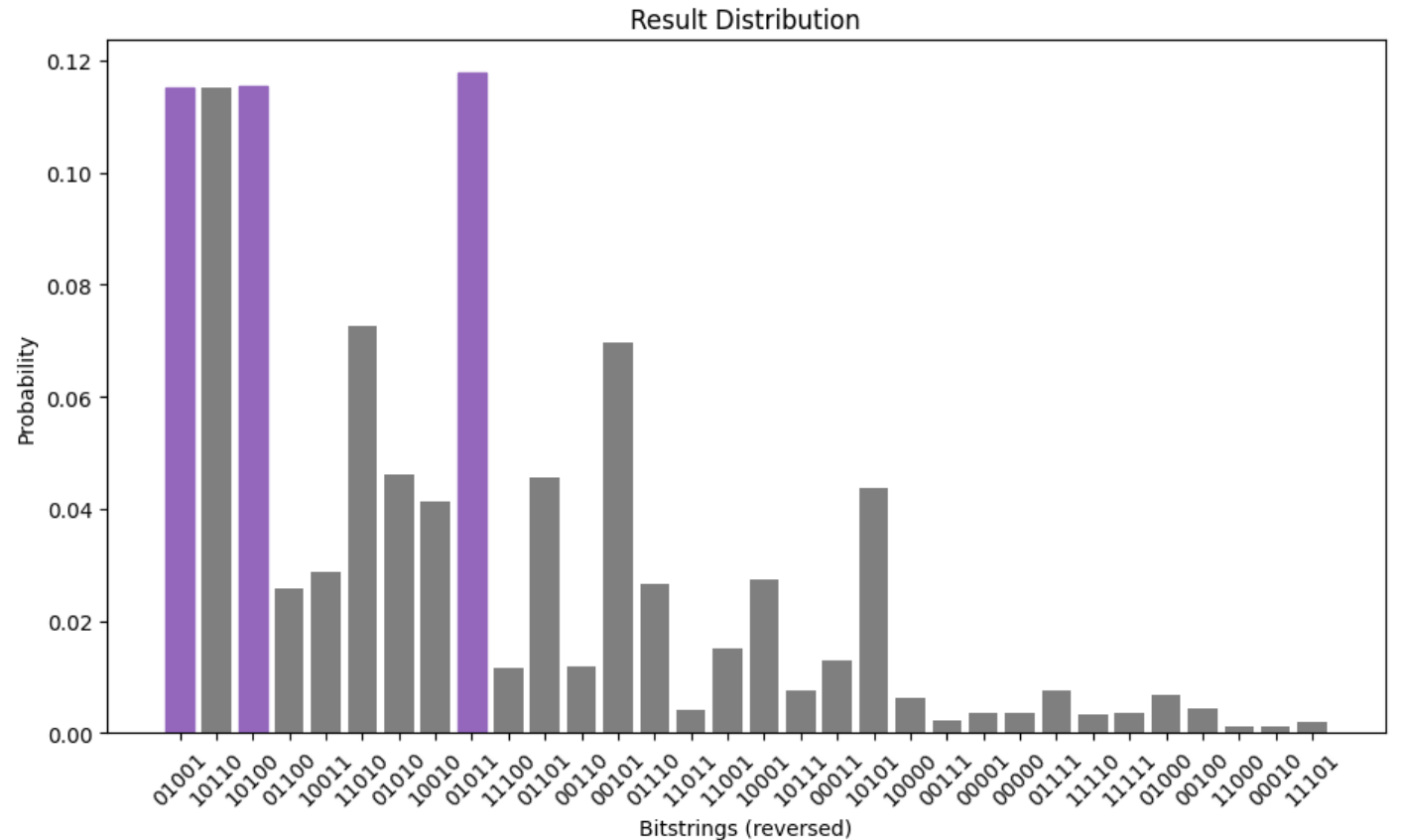
이전의 확률분포 데이터에서 가장 확률이 높은 비트열을 가공, 출력하기

```
def to_bitstring(integer, num_bits):  
    result = np.binary_repr(integer, width=num_bits) # 10진수를 2진수 비트열로 변환  
    return [int(digit) for digit in result]
```

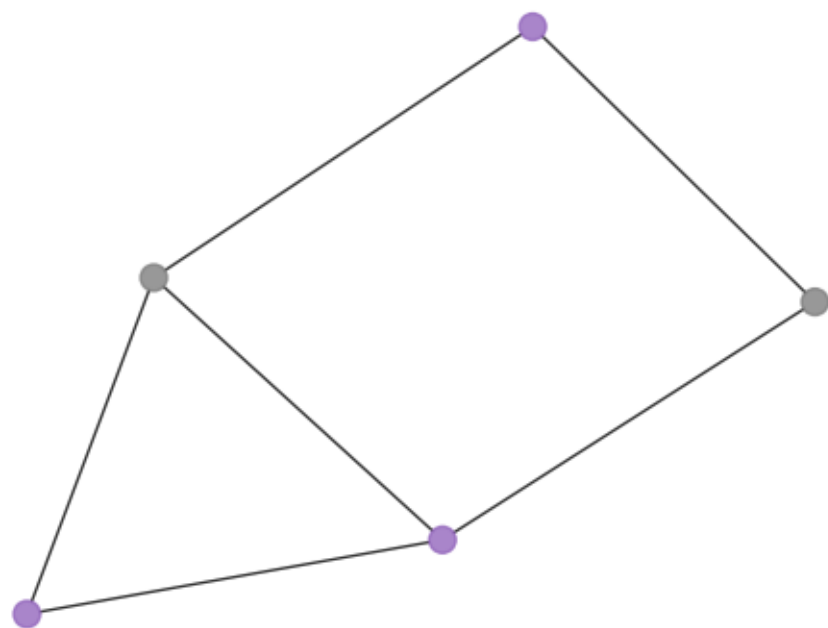
```
keys = list(final_distribution_int.keys())  
values = list(final_distribution_int.values())  
most_likely = keys[np.argmax(np.abs(values))] # 가장 높은 확률의 데이터 추출  
most_likely_bitstring = to_bitstring(most_likely, len(graph))  
most_likely_bitstring.reverse() # 리틀 엔디안 되돌리기
```

```
print("Result bitstring:", most_likely_bitstring)
```

Result bitstring: [1, 1, 0, 1, 0]



4. 결과 해석



...

Classical optimum (brute force): 5

QAOA cut value: 5

감사합니다.